



รายงาน
ระบบให้อาหารสัตว์เลี้ยงอัตโนมัติ
Automatic Pet Feeder System

จัดทำโดย
นางสาว สุดาร์ตน์ สุขอยู่

เสนอ
รศ.ดร. สิทธิชัย ชูสำโรง

รายงานนี้เป็นส่วนหนึ่งของรายวิชา Basic Internet of Things
(พื้นฐานอินเทอร์เน็ตของสรรพสิ่ง) รหัสวิชา 104333
ภาคเรียนที่ 2 ปีการศึกษา 2568

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญ

ในปัจจุบันการเลี้ยงสัตว์เลี้ยงถือเป็นกิจกรรมที่ได้รับความนิยมอย่างแพร่หลาย เจ้าของสัตว์เลี้ยงหลายคนต้องเผชิญกับปัญหาเรื่องการให้อาหารอย่างสม่ำเสมอ โดยเฉพาะในกรณีที่มีภารกิจยุ่งหรือต้องเดินทางออกนอกบ้านเป็นเวลานาน

โครงการนี้จึงได้พัฒนาระบบให้อาหารสัตว์เลี้ยงอัตโนมัติ โดยใช้ไมโครคอนโทรลเลอร์ ESP8266 ร่วมกับแพลตฟอร์ม IoT ชื่อ Blynk เพื่อให้ผู้ใช้สามารถควบคุมและตรวจสอบการให้อาหารได้จากระยะไกลผ่านสมาร์ทโฟน

1.2 วัตถุประสงค์ของโครงการ

- เพื่อออกแบบและพัฒนาระบบให้อาหารสัตว์เลี้ยงอัตโนมัติที่ควบคุมผ่านอินเทอร์เน็ต (IoT)
- เพื่อให้ผู้ใช้สามารถตั้งเวลาให้อาหารได้ 2 รอบต่อวัน (เช้าและเย็น) ผ่านแอปพลิเคชัน Blynk
- เพื่อรองรับการให้อาหารแบบ Manual ผ่านปุ่มในแอป และผ่านบัตร RFID
- เพื่อแจ้งเตือนเมื่ออาหารใกล้หมดผ่านการแจ้งเตือนในแอปพลิเคชัน
- เพื่อจำกัดจำนวนครั้งในการให้อาหารต่อวัน เพื่อป้องกันการให้อาหารเกินขนาด

1.3 ขอบเขตของโครงการ

- ระบบทำงานบน ESP8266 (NodeMCU) ที่เชื่อมต่อกับ Wi-Fi และ Blynk Cloud
- ควบคุม Servo Motor เปิด-ปิดช่องปล่อยอาหาร
- อ่านบัตร RFID ด้วย MFRC522 สำหรับการให้อาหารแบบ Physical
- ตรวจจกระดับอาหารด้วย IR Sensor และส่งแจ้งเตือนไปยังแอป
- จำกัดจำนวนการให้อาหารสูงสุด 3 ครั้งต่อวัน

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- ช่วยให้เจ้าของสัตว์เลี้ยงสามารถให้อาหารได้ตรงเวลาแม้อยู่ห่างบ้าน
- ลดความเสี่ยงจากการลืมให้อาหารสัตว์เลี้ยง
- ได้รับการแจ้งเตือนเมื่ออาหารใกล้หมด เพื่อเติมได้ทันเวลา
- สามารถขยายระบบเพิ่มเติมในอนาคตได้ เช่น กล้อง หรือการชั่งน้ำหนักอาหาร

บทที่ 2 อุปกรณ์ที่ใช้ในโครงการ

2.1 รายการอุปกรณ์

โครงการนี้ใช้อุปกรณ์ฮาร์ดแวร์และซอฟต์แวร์ดังต่อไปนี้

อุปกรณ์	จำนวน	หน้าที่
ESP8266 (NodeMCU)	1	ไมโครคอนโทรลเลอร์หลัก ควบคุมการทำงานทั้งหมด เชื่อมต่อ Wi-Fi
Micro Servo Motor SG 180	1	เปิด-ปิดช่องปล่อยอาหาร
RFID Module (MFRC522)	1	อ่านบัตร RFID เพื่อสั่งให้อาหาร
IR Sensor	1	ตรวจจับระดับอาหารในถัง
แอปพลิเคชัน Blynk	1	แสดงสถานะ รับการแจ้งเตือน ควบคุมผ่านมือถือ
Wi-Fi Router	1	เชื่อมต่ออินเทอร์เน็ตสำหรับ ESP8266 และ Blynk
แหล่งจ่ายไฟ 18V	1	จ่ายไฟให้แก่วงจรทั้งหมด

2.2 รายละเอียดอุปกรณ์หลัก

2.2.1 ESP8266 (NodeMCU)

ESP8266 เป็นไมโครคอนโทรลเลอร์ที่มีโมดูล Wi-Fi ในตัว ทำงานที่แรงดัน 3.3V มี GPIO หลายขา รองรับการสื่อสาร SPI, I2C, UART และสามารถโปรแกรมผ่าน Arduino IDE ได้ เหมาะสำหรับงาน IoT ที่ต้องการเชื่อมต่ออินเทอร์เน็ต

2.2.2 Micro Servo Motor SG 180 Degree

Servo Motor ใช้สำหรับควบคุมการเปิด-ปิดช่องปล่อยอาหาร รับสัญญาณ PWM จาก ESP8266 ผ่านขา D4 สามารถกำหนดมุมได้ 0-180 องศา ในโครงการนี้ใช้มุม 0 องศา (ปิด) และ 140 องศา (เปิด)

2.2.3 RFID Module (MFRC522)

โมดูล MFRC522 ใช้สำหรับอ่านบัตร RFID ความถี่ 13.56 MHz สื่อสารกับ ESP8266 ผ่านโปรโตคอล SPI ช่วยให้ผู้ใช้สามารถสั่งให้อาหารได้โดยการแตะบัตรที่กำหนดไว้

2.2.4 IR Sensor

เซนเซอร์อินฟราเรดใช้ตรวจจับว่ายังมีอาหารอยู่หรือไม่ เมื่อไม่มีอาหารขวางแสงอินฟราเรด เซนเซอร์จะส่งสัญญาณ HIGH บอกให้ระบบทราบว่าอาหารใกล้หมด

2.2.5 Blynk IoT Platform

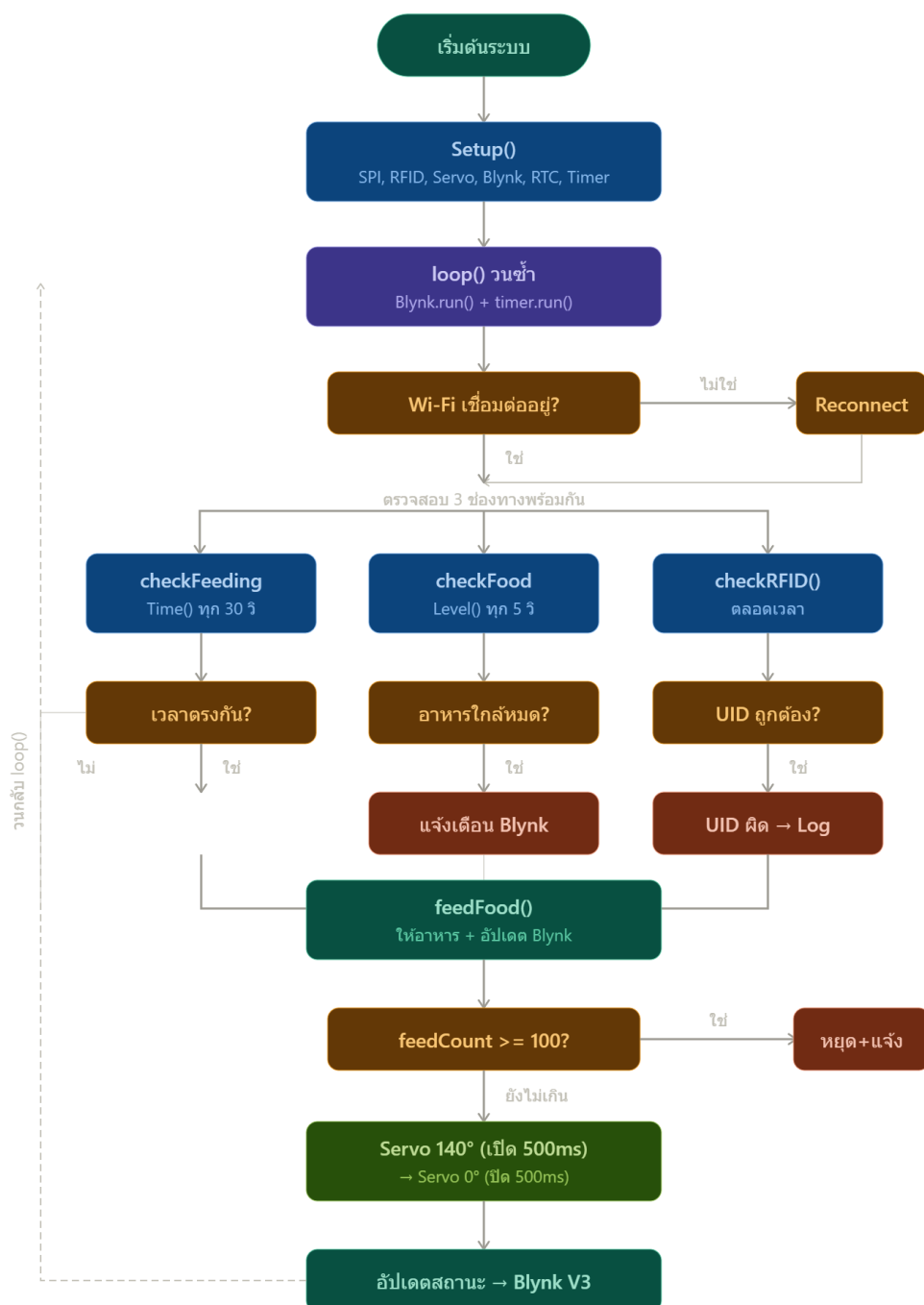
Blynk เป็นแพลตฟอร์ม IoT ที่ให้บริการ Cloud Server และแอปพลิเคชันมือถือสำหรับ iOS และ Android ในโครงการนี้ใช้ Virtual Pin V0-V4 สำหรับปุ่มควบคุม ตั้งเวลา และแสดงสถานะ

บทที่ 3 ลำดับขั้นตอนการทำงาน (Workflow)

3.1 ภาพรวมการทำงานของระบบ

ระบบให้อาหารอัตโนมัติทำงานโดยมีลำดับขั้นตอนหลัก 4 เส้นทาง ได้แก่ การให้อาหารตามเวลาที่กำหนด การให้อาหารด้วยปุ่ม Manual การให้อาหารผ่าน RFID และการแจ้งเตือนอาหารใกล้หมด

3.2 แผนภาพลำดับขั้นตอนการทำงาน (Flowchart)



3.3 คำอธิบายขั้นตอนการทำงาน

3.3.1 การเริ่มต้นระบบ (Setup)

- ระบบเริ่มต้น Serial Communication ความเร็ว 9600 bps
- เริ่มต้น SPI Bus และ RFID Module (MFRC522)
- กำหนดขา Servo (D4) และ IR Sensor (D1) เป็น Input/Output
- ตั้งตำแหน่ง Servo เริ่มต้นที่ 0 องศา (ปิดช่องอาหาร)
- เชื่อมต่อ Blynk Cloud ผ่าน Wi-Fi
- เริ่มต้น RTC Widget เพื่อซิงค์เวลาจาก NTP Server
- ตั้ง Timer ตรวจสอบเวลาให้อาหารทุก 30 วินาที และตรวจสอบระดับอาหารทุก 5 วินาที

3.3.2 การทำงานในลูปหลัก (Loop)

- ตรวจสอบการเชื่อมต่อ Wi-Fi ถ้าหลุดจะพยายามเชื่อมต่อใหม่
- เรียก Blynk.run() รับ-ส่งข้อมูลกับ Blynk Cloud
- เรียก timer.run() ทำงานตาม Interval ที่กำหนด
- เรียก checkRFID() ตรวจสอบการแตะบัตรตลอดเวลา

3.3.3 การให้อาหารตามเวลา (Scheduled Feeding)

- ระบบอ่านเวลาปัจจุบันจาก RTC Widget ทุก 30 วินาที
- เปรียบเทียบกับเวลาที่ผู้ใช้กำหนดไว้ใน V1 (เช้า) และ V2 (เย็น)
- ถ้าเวลาตรงกันและยังไม่ได้ให้อาหารในรอบนั้น เรียก feedFood()
- ใช้ Flag (fedMorning / fedEvening) ป้องกันการให้อาหารซ้ำในนาทีเดียวกัน
- รีเซ็ต Flag ทุกวันเวลา 00:00 น. และรีเซ็ตนับจำนวนครั้งเป็น 0

3.3.4 ฟังก์ชันให้อาหาร (feedFood)

- ตรวจสอบว่าจำนวนครั้งการให้อาหารเกิน 100 ครั้งหรือไม่ ถ้าใช่จะหยุดทำงานและแจ้งเตือน
- เพิ่มตัวนับ feedCount ขึ้น 1
- แสดงข้อความ 'กำลังให้อาหาร' บน Blynk V3
- สั่ง Servo หมุนไป 140 องศา (เปิดช่องอาหาร) รอ 500ms
- สั่ง Servo กลับมา 0 องศา (ปิดช่องอาหาร) รอ 500ms
- แสดงข้อความสรุปจำนวนครั้งที่ให้อาหารแล้วบน Blynk V3

บทที่ 4 การต่อสายวงจร

4.1 ตารางการต่อสาย (Wiring Table)

การเชื่อมต่อสายระหว่าง ESP8266 กับอุปกรณ์ต่างๆ มีรายละเอียดดังนี้

ESP8266 Pin	เชื่อมต่อกับ	หมายเหตุ
D1	IR Sensor (OUT)	รับสัญญาณดิจิทัล HIGH/LOW
D4	Micro Servo Motor SG 180 (Signal)	ส่งสัญญาณ PWM ควบคุม Servo
D8 (SS)	MFRC522 (SDA/SS)	SPI Chip Select
D3	MFRC522 (RST)	Reset RFID Module
D5 (SCK)	MFRC522 (SCK)	SPI Clock
D6 (MISO)	MFRC522 (MISO)	SPI Data Out จาก Module
D7 (MOSI)	MFRC522 (MOSI)	SPI Data In ไปยัง Module
3.3V	MFRC522 (VCC), IR Sensor (VCC)	จ่ายไฟ 3.3V
GND	MFRC522 (GND), IR (GND), Servo (GND)	กราวด์
5V (VIN)	Servo Motor (VCC)	จ่ายไฟ 5V ให้ Servo

4.2 คำอธิบายการต่อวงจร

4.2.1 การต่อ RFID Module (MFRC522)

MFRC522 ใช้การสื่อสารแบบ SPI จึงต้องต่อขา SCK, MISO, MOSI ร่วมกันบน SPI Bus ของ ESP8266 นอกจากนี้ขา SDA (SS) ต่อกับ D8 และขา RST ต่อกับ D3 สำหรับ VCC ใช้แรงดัน 3.3V เท่านั้น ห้ามต่อกับ 5V โดยเด็ดขาดเพราะจะทำให้ไมโครคอนโทรลเลอร์เสียหาย

4.2.2 การต่อ Micro Servo Motor SG 180 Degree

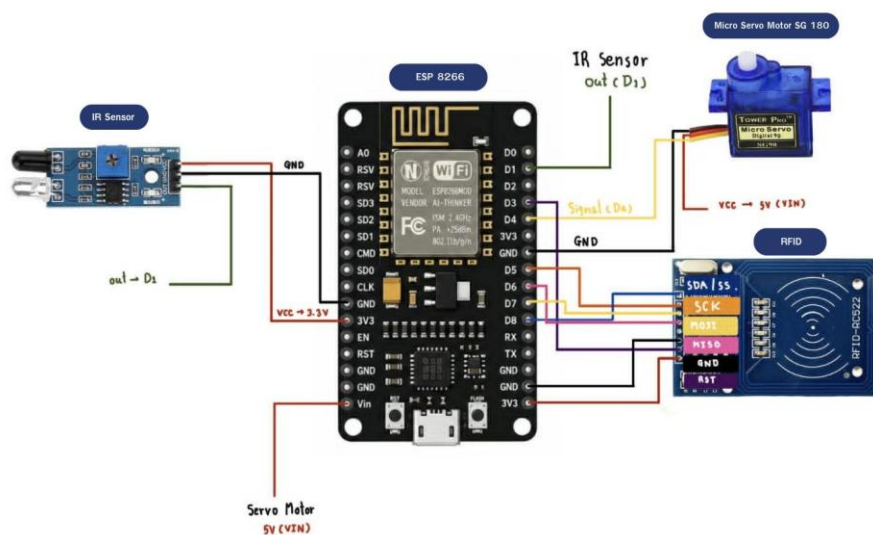
Micro Servo Motor ต้องการแรงดัน 5V สำหรับ VCC (ต่อกับขา VIN ของ NodeMCU) สายสัญญาณ PWM ต่อกับ D4 และ GND ต่อร่วมกับกราวด์ของวงจร ระวังการใช้กระแสไฟสูงอาจทำให้แรงดันตก ควรใช้ Capacitor 100μF ต่อคร่อม VCC-GND ของ Servo

4.2.3 การต่อ IR Sensor

IR Sensor มี 3 ขาคือ VCC, GND และ OUT ขา VCC ต่อ 3.3V หรือ 5V (ขึ้นอยู่กับรุ่น) ขา OUT ต่อกับ D1 ของ ESP8266 ปรับความไวของเซนเซอร์ด้วยตัวต้านทาน Trim Pot บนโมดูล ให้ระยะการตรวจจับประมาณ 2-5 เซนติเมตร

4.3 แผนภาพวงจร (Circuit Diagram)

แผนภาพวงจรแสดงการเชื่อมต่อระหว่างอุปกรณ์ทั้งหมด (ดูรูปที่ 4.1)



รูปที่ 4.1 แผนภาพวงจรระบบให้อาหารอัตโนมัติ

บทที่ 5 โค้ดโปรแกรมและคำอธิบาย

5.1 ตารางสรุปหน้าที่ของแต่ละฟังก์ชัน

ชื่อฟังก์ชัน / ส่วน	หน้าที่
#define & ค่าคงที่	กำหนด Template ID, Auth Token, ขาพิน IR และ RFID
feedFood()	ฟังก์ชันหลักสำหรับให้อาหาร: เช็กจำนวนครั้ง (max 100), สั่ง Servo เปิด-ปิด, อัปเดตสถานะผ่าน Blynk V3
BLYNK_WRITE(V0)	รับคำสั่งจากปุ่ม Manual บน Blynk App เมื่อกด จะเรียก feedFood()
BLYNK_WRITE(V1)	รับค่าเวลาอาหารเข้าจาก Time Input Widget บน Blynk (เก็บชั่วโมง + นาที)
BLYNK_WRITE(V2)	รับค่าเวลาอาหารเย็นจาก Time Input Widget บน Blynk (เก็บชั่วโมง + นาที)
checkFeedingTime()	เช็กเวลาปัจจุบัน (NTP ผ่าน WidgetRTC) เทียบกับเวลาที่กำหนด ถ้าถึงเวลาจะเรียก feedFood() พร้อม flag กันซ้ำ รีเซ็ตทุกเที่ยงคืน
checkFoodLevel()	อ่านค่า IR Sensor HIGH = อาหารใกล้หมด ส่ง Notification และ logEvent ไป Blynk
checkRFID()	อ่าน UID จาก RFID Card ถ้าตรงกับ UID ที่กำหนด จะเรียก feedFood() พร้อม logEvent; ถ้าไม่ตรงบันทึก wrong_card
setup()	เริ่มต้น Serial, SPI, RFID, Servo, GPIO, เชื่อมต่อ Blynk, ตั้ง Timer
loop()	วนซ้ำ: เช็ก Wi-Fi, รัน Blynk.run(), timer.run(), checkRFID()

5.2 โค้ดโปรแกรมทั้งหมดพร้อมคำอธิบาย

5.2.1 ส่วนที่ 1 การกำหนดค่าและไลบรารี

ส่วนนี้กำหนดค่า Template ID และ Auth Token ของ Blynk, กำหนดขาพิน และ include ไลบรารีที่จำเป็น

```
#define BLYNK_TEMPLATE_ID "TMPL62kDSntMx"
#define BLYNK_TEMPLATE_NAME "Feed System"
#define BLYNK_AUTH_TOKEN "Dw_EKwhXTJ2UyalhvdnqkvyX3fJz8JUE"
#define BLYNK_PRINT Serial

#define IR_SENSOR D1    // ขา IR Sensor
#define SS_PIN D8       // ขา SDA ของ MFRC522
#define RST_PIN D3      // ขา RST ของ MFRC522
```



```
#include <ESP8266WiFi.h>      // Wi-Fi สำหรับ ESP8266
#include <BlynkSimpleEsp8266.h> // Blynk IoT Library
#include <Servo.h>             // ควบคุม Servo Motor
#include <TimeLib.h>           // จัดการเวลา
#include <WidgetRTC.h>         // Real-Time Clock Widget ของ Blynk
#include <SPI.h>               // SPI Communication
#include <MFRC522.h>           // RFID Module

char ssid[] = "sudaratsuko"; // ชื่อ Wi-Fi
char pass[] = "22222222";     // รหัส Wi-Fi
```

5.2.2 ส่วนที่ 2 ตัวแปรและออบเจกต์

ประกาศออบเจกต์สำหรับ Micro Servo, Blynk Timer, RTC และ RFID รวมถึงตัวแปรสำหรับเก็บเวลาและสถานะการให้อาหาร

```
Servo myservo;      // ออบเจกต์ควบคุม Servo
BlynkTimer timer;   // Timer สำหรับเรียกฟังก์ชันซ้ำตาม Interval
WidgetRTC rtc;      // RTC Widget ดึงเวลาจาก NTP ผ่าน Blynk
MFRC522 rfid(SS_PIN, RST_PIN); // RFID Module

int startHour1, startMinute1; // เวลาอาหารเช้า (ชั่วโมง, นาที)
int startHour2, startMinute2; // เวลาอาหารเย็น (ชั่วโมง, นาที)
bool fedMorning = false;      // Flag: ให้อาหารเช้าแล้วหรือยัง
bool fedEvening = false;     // Flag: ให้อาหารเย็นแล้วหรือยัง
int feedCount = 0;           // นับจำนวนครั้งที่ให้อาหารวันนี้
```

5.2.3 ส่วนที่ 3 ฟังก์ชัน feedFood()

ฟังก์ชันหลักในการให้อาหาร ตรวจสอบจำนวนครั้ง สั่ง Servo และส่งสถานะไป Blynk

```
void feedFood() {
    // ตรวจสอบจำนวนครั้ง ถ้าเกิน 100 ครั้ง หยุดและแจ้งเตือน
    if (feedCount >= 100) {
        Blynk.virtualWrite(V3, "วันนี้ให้อาหารครบ 3 ครั้งแล้ว");
        Blynk.logEvent("feed_limit", "วันนี้ให้อาหารครบ 3 ครั้งแล้ว");
        return;
    }
}
```

```

feedCount++; // เพิ่มตัวนับ

// อัปเดตสถานะบน Blynk
Blynk.virtualWrite(V3, "กำลังให้อาหาร\นครั้งที่ " + String(feedCount));

myservo.write(140); // เปิดช่องอาหาร (หมุน Servo 140 องศา)
delay(500);
myservo.write(0); // ปิดช่องอาหาร (หมุน Servo กลับ 0 องศา)
delay(500);

// แสดงสรุปจำนวนครั้งที่ให้อาหาร
Blynk.virtualWrite(V3, "ให้อาหารแล้ว\นรวม " + String(feedCount) + " ครั้ง");
}

```

5.2.4 ส่วนที่ 4: Blynk Virtual Pins Handler

ฟังก์ชัน BLYNK_WRITE รับค่าจาก Virtual Pin ต่างๆ บน Blynk App

```

// V0: ปุ่ม Manual บน Blynk App
BLYNK_WRITE(V0) {
  if (param.asInt() == 1) // เมื่อกดปุ่ม
    feedFood();
}

// V1: Time Input Widget สำหรับเวลาอาหารเช้า
BLYNK_WRITE(V1) {
  TimeInputParam t(param);
  if (t.hasStartTime()) {
    startHour1 = t.getStartHour();
    startMinute1 = t.getStartMinute();
  }
}

// V2: Time Input Widget สำหรับเวลาอาหารเย็น
BLYNK_WRITE(V2) {
  TimeInputParam t(param);

```

```

if (t.hasStartTime()) {
    startHour2 = t.getStartHour();
    startMinute2 = t.getStartMinute();
}
}

```

5.2.5 ส่วนที่ 5 ฟังก์ชัน checkFeedingTime()

เช็ควเวลาปัจจุบันเทียบกับเวลาที่กำหนด พร้อมรีเซ็ตทุกเที่ยงคืน

```

void checkFeedingTime() {
    int currentHour = hour();    // อ่านชั่วโมงจาก RTC
    int currentMinute = minute(); // อ่านนาทีจาก RTC

    // รอบเช้า: ถ้าเวลาตรงและยังไม่ได้ให้อาหาร
    if (currentHour == startHour1 && currentMinute == startMinute1 && !fedMorning) {
        feedFood();
        fedMorning = true; // ตั้ง Flag กันให้อาหารซ้ำ
    }

    // รอบเย็น: ถ้าเวลาตรงและยังไม่ได้ให้อาหาร
    if (currentHour == startHour2 && currentMinute == startMinute2 && !fedEvening) {
        feedFood();
        fedEvening = true;
    }

    // รีเซ็ตทุกวันตอนเที่ยงคืน (00:00)
    if (currentHour == 0 && currentMinute == 0) {
        fedMorning = false;
        fedEvening = false;
        feedCount = 0;
    }
}

```

5.2.6 ส่วนที่ 6 ฟังก์ชัน checkFoodLevel()

ตรวจสอบระดับอาหารด้วย IR Sensor และแจ้งเตือนเมื่ออาหารใกล้หมด

```
void checkFoodLevel() {
  int sensorValue = digitalRead(IR_SENSOR);

  if (sensorValue == HIGH) { // HIGH = ไม่เจออาหาร (อาหารใกล้หมด)
    Blynk.virtualWrite(V4, "อาหารใกล้หมดแล้ว!!");
    Blynk.logEvent("food_low", "อาหารใกล้หมดแล้ว!!");
  } else { // LOW = มีอาหารปกติ
    Blynk.virtualWrite(V4, "อาหารปกติ");
  }
}
```

5.2.7 ส่วนที่ 7 ฟังก์ชัน checkRFID()

อ่านบัตร RFID และตรวจสอบ UID ว่าตรงกับที่กำหนดไว้หรือไม่

```
void checkRFID() {
  if (!rfid.PICC_IsNewCardPresent()) return; // ไม่มีการ์ด
  if (!rfid.PICC_ReadCardSerial()) return; // อ่านไม่ได้

  // รวม UID เป็น String
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++)
    uid += String(rfid.uid.uidByte[i], HEX);

  if (uid == "7e73b36") { // UID ที่ได้รับอนุญาต
    Blynk.virtualWrite(V3, "RFID ให้อาหาร");
    feedFood();
    Blynk.logEvent("feed_alert", "มีการให้อาหารผ่าน RFID");
  } else { // UID ที่ไม่ได้รับอนุญาต
    Blynk.logEvent("wrong_card", "พบบัตรที่ไม่ได้รับอนุญาต");
  }
  rfid.PICC_HaltA(); // หยุดการสื่อสารกับการ์ด
}
```

5.2.8 ส่วนที่ 8 setup() และ loop()

ฟังก์ชัน setup() เริ่มต้นระบบทั้งหมด และ loop() วนซ้ำตลอดการทำงาน

```
void setup() {
  Serial.begin(9600);
  SPI.begin();          // เริ่มต้น SPI Bus
  rfid.PCD_Init();      // เริ่มต้น RFID Module
  myservo.attach(D4);   // กำหนดขา Servo
  pinMode(IR_SENSOR, INPUT);
  myservo.write(0);      // ตำแหน่งเริ่มต้น (ปิดช่องอาหาร)
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass); // เชื่อมต่อ Blynk
  rtc.begin();          // เริ่มต้น RTC Widget
  // ตั้ง Timer
  timer.setInterval(30000L, checkFeedingTime); // เช็เวลาทุก 30 วินาที
  timer.setInterval(5000L, checkFoodLevel);    // เช็คอาหารทุก 5 วินาที
}

void loop() {
  // เช็ก Wi-Fi ถ้าหลุดให้เชื่อมต่อใหม่
  if (WiFi.status() != WL_CONNECTED)
    WiFi.begin(ssid, pass);

  Blynk.run(); // รับ-ส่งข้อมูลกับ Blynk Cloud
  timer.run(); // ทำงานตาม Timer ที่กำหนด
  checkRFID(); // ตรวจสอบ RFID Card ตลอดเวลา
}
```

5.3 Virtual Pin Mapping บน Blynk App

Virtual Pin	Widget	หน้าที่
V0	Button	ปุ่ม Manual สำหรับให้อาหารทันที
V1	Time Input	กำหนดเวลาอาหารเช้า
V2	Time Input	กำหนดเวลาอาหารเย็น
V3	Label / Display	แสดงสถานะการให้อาหาร
V4	Label / Display	แสดงสถานะระดับอาหาร

บทที่ 6 สรุปและข้อเสนอแนะ

6.1 สรุปผลการดำเนินงาน

โครงการระบบให้อาหารอัตโนมัติสามารถพัฒนาได้สำเร็จตามวัตถุประสงค์ที่วางไว้ ระบบสามารถทำงานได้ทั้งแบบให้อาหารตามเวลาที่กำหนด การให้อาหารด้วยปุ่มกดบนแอปพลิเคชัน และการให้อาหารผ่านบัตร RFID พร้อมทั้งแจ้งเตือนผู้ใช้เมื่ออาหารใกล้หมด

6.2 ปัญหาและแนวทางแก้ไข

- ปัญหา: Wi-Fi หลุดบ่อย แก้ไขโดยเพิ่มการเช็คและเชื่อมต่อใหม่อัตโนมัติใน loop()
- ปัญหา: Servo สั่นตอนไม่มีสัญญาณ แนะนำให้ใช้คำสั่ง `myservo.detach()` หลังใช้งาน
- ปัญหา: RTC อาจไม่ซิงค์เวลาที่บันทึก Blynk เชื่อมต่อสมบูรณ์ก่อนใช้ค่าเวลา

6.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ

- เพิ่มเซนเซอร์ชั่งน้ำหนักอาหาร (Load Cell) เพื่อวัดปริมาณอาหารที่ให้ได้แม่นยำขึ้น
- เพิ่มกล้อง IP Camera เพื่อให้ผู้ใช้ดูสัตว์เลี้ยงจากระยะไกลผ่านแอป
- พัฒนาระบบบันทึกประวัติการให้อาหาร (Feeding Log) บน Firebase หรือ Database
- รองรับ RFID หลายใบ และสามารถลงทะเบียน/ยกเลิกผ่านแอปได้
- เพิ่มระบบ OTA (Over-The-Air) Update สำหรับอัปเดตโปรแกรมโดยไม่ต้องเสียบสาย

6.4 บทสรุป

โครงการนี้แสดงให้เห็นถึงการประยุกต์ใช้เทคโนโลยี IoT ในชีวิตประจำวัน โดยการนำ ESP8266 มาเป็นศูนย์กลางควบคุม ร่วมกับ Blynk Platform ที่ช่วยให้การพัฒนา IoT Application ทำได้ง่ายและรวดเร็ว ระบบนี้สามารถนำไปต่อยอดและพัฒนาเพิ่มเติมได้อีกมากในอนาคต